

Relatório — Camada de Banco de Dados

Projeto Pokédex (trabalho em grupo)

Aluno: Luís Leão

Componente individual: Banco de Dados (SQLite + ORMLite)

Disciplina / Professor: POO/AKIRA_____

Data: junho de 2026

1. Objetivo

Este relatório descreve a camada de banco de dados do projeto Pokédex, de minha responsabilidade. O objetivo foi construir, a partir de um arquivo CSV com os 151 Pokémon da primeira geração, um banco de dados relacional que sirva de fonte de dados para as demais partes do sistema (as telas da aplicação e as classes de modelagem). A persistência foi feita com ORMLite, seguindo o padrão de camada de modelo (MVC) e o padrão Repository apresentados no tutorial da disciplina.

2. Tecnologias utilizadas

- SQLite — banco de dados em arquivo único, sem necessidade de instalação de servidor.
- ORMLite 6.1 — mapeamento objeto-relacional (ORM); a persistência é feita por métodos (create, query) em vez de SQL escrito à mão.
- Driver sqlite-jdbc 3.46.0.0 — conexão JDBC com o SQLite, usado por baixo pelo ORMLite.
- SLF4J (api e simple) 2.0.13 — biblioteca de log exigida pelo ORMLite.
- BlueJ — ambiente de desenvolvimento, com as bibliotecas adicionadas via Preferências › Bibliotecas.

3. Arquitetura adotada

A camada segue três tipos de classe, conforme o tutorial:

- Classe Database: gerencia a conexão com o arquivo SQLite (abrir e fechar).
- Classes entidade: POJOs anotados com @DatabaseTable e @DatabaseField, que representam as tabelas. Cada objeto corresponde a uma linha.
- Classes Repository: uma por entidade, com as operações sobre o banco (create, loadFromId, loadAll e consultas). As demais partes do sistema acessam os dados apenas pelos repositórios, sem escrever SQL.

4. Modelagem das entidades

Optou-se por entidades planas (sem tabelas de ligação), aderentes ao padrão do tutorial e ao formato do CSV de origem, que já trazia as colunas de tipo. As entidades e suas tabelas:

Entidade	Tabela	Conteúdo
Pokemon	pokemon	id (nº da Pokédex), nome, hp, ataque, defesa, sp_atk, sp_def, velocidade, sprite_url, tipo_1, tipo_2 e golpe.
Golpe	golpe	Catálogo de golpes: nome, tipo, poder, precisão e categoria.

Entidade	Tabela	Conteúdo
Nature	nature	As 25 natures: nome, atributo que aumenta e atributo que reduz.
TipoEfetividade	tipo_efetividade	Tabela de efetividade entre tipos (TypeChart): tipo atacante, tipo defensor e multiplicador (2.0, 0.5 ou 0.0).

Para cada entidade existe um repositório correspondente (PokemonRepository, GolpeRepository, NatureRepository e TipoEfetividadeRepository). O PokemonRepository possui, além do CRUD básico, consultas específicas do projeto: loadByName (busca por nome), loadByType (filtro por tipo) e fraquezasDe (cálculo das fraquezas e vantagens de um Pokémon, usado pelo painel de análise).

5. Origem e tratamento dos dados

A base foi um arquivo CSV com os 151 Pokémon da primeira geração (número, nome, dois tipos, seis atributos e a URL da imagem), obtido de um repositório público no GitHub. Sobre essa base foram feitos os seguintes tratamentos e complementos:

- Padronização do cabeçalho do CSV (nomes de coluna sem espaços nem acentos) e leitura em UTF-8, preservando caracteres especiais (por exemplo, Nidoran♀ e Nidoran♂).
- Tipo secundário ausente passa a ser nulo no banco, em vez de texto vazio.
- Atribuição de um golpe característico a cada Pokémon (uso visual, exibido no card; não há sistema de combate).
- Inclusão de dados canônicos que não constavam no CSV: a tabela de efetividade entre os 18 tipos (apenas os confrontos não neutros) e as 25 natures.

6. Processo de desenvolvimento

1. Definição do escopo e da fonte de dados (o CSV dos 151 Pokémon).
2. Tratamento do CSV (cabeçalho padronizado e codificação UTF-8).
3. Escolha do padrão de persistência (ORMLite + Repository, conforme o tutorial da disciplina).
4. Modelagem das entidades e definição das tabelas.
5. Implementação dos repositórios (operações CRUD e consultas).
6. Complemento dos dados canônicos (golpe por Pokémon, tabela de tipos e natures).
7. Implementação da classe de carga (PopularBanco), que lê os CSVs e persiste tudo via ORMLite.
8. Validação dos dados gerados (contagens e verificação de integridade).

7. Resultado

Ao executar a classe de carga, é gerado o arquivo pokedex.db já populado. Os volumes obtidos:

Conteúdo	Quantidade
Pokémon cadastrados	151
Golpes no catálogo	91

Conteúdo	Quantidade
Natures	25
Confrontos na tabela de efetividade	120

A verificação confirmou que todos os Pokémon foram importados, que cada um possui exatamente um golpe associado e que não há violações de integridade referencial nos dados.

8. Integração com o restante do projeto

O banco é a camada de dados que alimenta as demais partes. A interface não acessa o banco diretamente nem escreve SQL: ela cria um objeto Database e os repositórios e, a partir deles, obtém objetos Pokémon prontos para exibição. Por exemplo, a grade de Pokémon usa `loadAll`, a barra de busca usa `loadByName`, o filtro usa `loadByType` e o painel de análise usa `fraquezasDe`. Dessa forma, a modelagem do banco corresponde diretamente às classes de modelo do projeto.

9. Como executar

9. Adicionar ao BlueJ as cinco bibliotecas (`sqlite-jdbc`, `ormlite-core`, `ormlite-jdbc`, `slf4j-api` e `slf4j-simple`).
10. Manter os três arquivos CSV na pasta do projeto.
11. Compilar as classes.
12. Executar o método `main` da classe `PopularBanco`, que cria e popula o arquivo `pokedex.db`.

10. Considerações finais

A camada de banco de dados está concluída e validada, seguindo o padrão ORM e Repository solicitado. Restam decisões de equipe que podem gerar pequenas extensões (todas no mesmo padrão), como permitir a edição de golpe ou nature na montagem do time e, caso necessário, a persistência do time montado pelo usuário.